

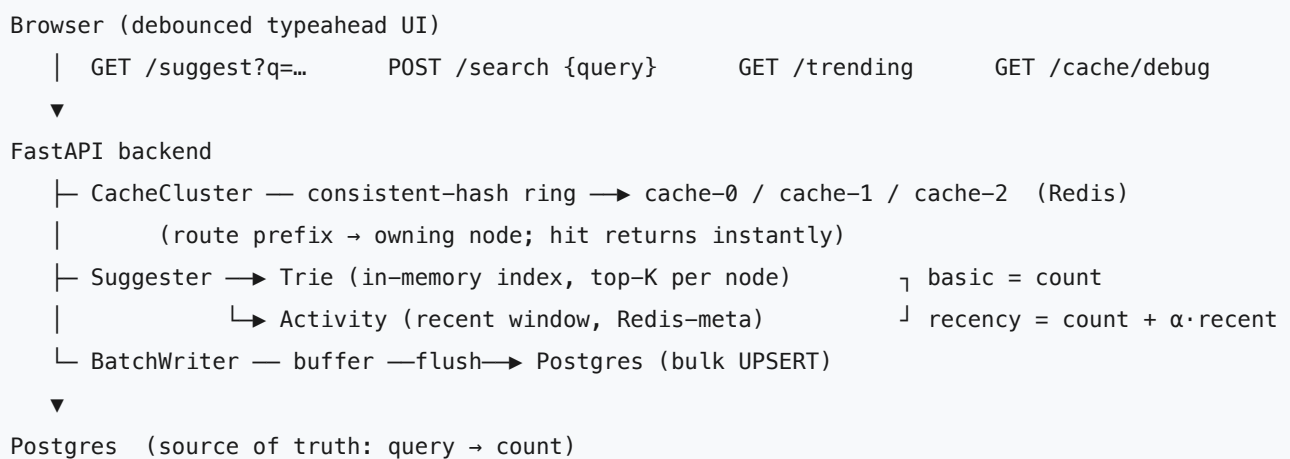
Search Typeahead System — Project Report

Student: Prateek Singh | **Email:** prateek.24bcs10082@sst.scaler.com

Repository: <https://github.com/PRAteek-singHWY/search-typeahead-system> **Evaluation commit:** [c14fd1728b27b2e21ba0c7690e84959af239f647](#)

A working search typeahead / autocomplete service: as the user types a prefix it returns the most popular matching queries, records submitted searches, updates query popularity, supports trending searches, and uses a distributed cache for low-latency reads. Stack: **Python / FastAPI + Postgres + 3 Redis cache nodes + Redis-meta**, all via Docker Compose.

1. Architecture



What it demonstrates (and where)

Concept	File	One-line explanation
Data modeling	app/db.py (Postgres)	One table: <code>search_queries(query, count, last_searched_at)</code> .
Suggestion engine	app/trie.py	A Trie where each node caches its subtree's top candidates → $O(\text{prefix})$ lookups.
Distributed cache	app/cache_cluster.py	3 Redis cache nodes; consistent hashing routes each prefix to one node.
Consistent hashing	app/consistent_hash.py	Hash ring with virtual nodes; adding a node remaps only $\sim 1/N$ of prefixes.
Trending / recency	app/activity.py , app/suggester.py	Sliding minute-window counters; recency-aware ranking blends popularity + recent activity.
Batch writes	app/batch_writer.py	Search events are buffered and flushed as one bulk UPSERT.

See [DESIGN.md](#) for the reasoning and trade-offs behind every choice, and [PERFORMANCE.md](#) for measured latency, cache hit rate, and write reduction.

How to run

You need Docker. Then:

```
docker compose up --build
```

This starts Postgres, three Redis cache nodes, a Redis-meta node, and the backend. On first startup it **generates a 100,000-query dataset** (`backend/data/queries.csv`), ingests it, and builds the Trie. When ready, open:

- **App UI:** <http://localhost:8000/>
- **API docs:** <http://localhost:8000/docs>

Stop and wipe everything (including the DB volume):

```
docker compose down -v
```

2. Dataset — source & loading

Dataset

- **Source:** synthetic, generated by `backend/scripts/generate_dataset.py` (fixed seed → reproducible). It produces $\geq 100,000$ unique queries with a heavy-tailed (Pareto) popularity distribution, in `query,count` CSV format.
- **Using your own dataset:** drop any `query,count` CSV at `backend/data/queries.csv` and restart with an empty DB (`docker compose down -v && docker compose up`). `app/db.py::ingest_csv` lowercases, aggregates duplicates, and bulk-loads it.

3. API documentation

API

API	Purpose	Behaviour
<code>GET /suggest?q=<prefix>&ranking=basic\ recency&limit=10</code>	Fetch suggestions	Up to 10 prefix-matches sorted by count (basic) or recency-aware score.
<code>POST /search {"query": "..."} </code>	Submit a search	Returns <code>{"message": "Searched"}</code> , records the query (batched write + trending).
<code>GET /cache/debug?prefix=<prefix></code>	Debug cache routing	Shows which cache node owns the prefix and whether it's currently cached (hit/miss).
<code>GET /trending?k=10</code>	Trending searches	Top queries in the recent sliding window.
<code>GET /metrics</code>	Performance counters	Cache hit rate, DB read/write counts, batch write-reduction factor.
<code>GET /stats</code>	Config + dataset size	Useful for the demo.

Quick API examples

```
curl "http://localhost:8000/suggest?q=iph&ranking=basic"
curl "http://localhost:8000/suggest?q=iph&ranking=recency"
curl -X POST http://localhost:8000/search -H "Content-Type: application/json" -d '{"query":"iph"
curl "http://localhost:8000/cache/debug?prefix=iph"
curl "http://localhost:8000/trending?k=10"
curl "http://localhost:8000/metrics"
```

4. Design choices & trade-offs

The following is the full design rationale (each section names the file that implements it).

Design Document — Search Typeahead

This explains *why* the system is built the way it is, so every major choice can be defended in a viva. Read it next to the code; each section names the file that implements it. Section numbers follow the assignment brief.

1. Problem & approach

When a user types a prefix we must return the most relevant completions almost instantly (typeahead must feel real-time). "Relevant" = **most popular** (highest search count), with an enhanced mode that also rewards **recent** activity. We also record submitted searches, surface **trending** queries, distribute the **cache** for low latency, and **batch writes** to protect the database.

The data path is: **Postgres** is the durable source of truth (query → count); an in-memory **Trie** is the fast suggestion index built from it; a **distributed Redis cache** sits in front of the Trie; **Redis-meta** holds the recent-activity window; and a **batch writer** absorbs write load.

2. Data modeling (`app/db.py`)

One table:

```
search_queries(
    query          TEXT PRIMARY KEY,
    count          BIGINT NOT NULL DEFAULT 0,
    last_searched_at TIMESTAMPTZ
)
```

- `query` as **primary key** → free unique index, and it enables an atomic "insert-or-increment" via `ON CONFLICT ... DO UPDATE` (the batch UPSERT).
- `count` is the popularity the suggestions rank by; `BIGINT` because popular queries grow large.
- `last_searched_at` aids debugging and could later drive decay.

Why one table is enough: suggestions are fully determined by (query, count). The Trie is the precomputed suggestion index and is rebuildable from this table on startup; trending lives in Redis (short-lived, high-write). No second table needed.

3. Dataset (`scripts/generate_dataset.py` , `app/db.py::ingest_csv`)

We generate $\geq 100,000$ unique queries with a **Pareto (heavy-tailed)** count distribution — a few very popular queries, a long tail of rare ones — which mirrors real search traffic and makes ranking/trending meaningful. The generator is seeded, so the dataset is reproducible. Any `query, count` CSV can be substituted; ingestion lowercases, aggregates duplicates, and bulk-loads in chunks.

4. The suggestion engine: Trie with cached top-K (`app/trie.py`)

A **Trie** stores queries along character paths, so every node is a prefix and the subtree below holds all completions of that prefix.

Key optimisation: each node caches the **top candidate pool** of its subtree, so a query is "walk to the prefix node, read its cached list" → **$O(\text{length of prefix})$** , independent of how many queries match. We cache a *pool* (30) slightly larger than the 10 we return, so the recency re-ranking has spare candidates.

Two build paths: - `bulk_build` (startup): insert all words, then compute each node's pool in one bottom-up pass. Building 100k queries takes ~2s — far faster than maintaining the pool on every insert. - `insert` (live updates from the batch writer): maintain the pool incrementally along the path. Correct because counts only ever increase, so a re-inserted query with a higher count simply re-sorts into place; a query that had dropped out can climb back in when its count rises.

Why a Trie and not `... WHERE query LIKE 'pre%'` ? The SQL form scans and sorts at query time and hits the DB on every keystroke. The Trie keeps the hot path in memory and exploits the prefix structure. Cost: extra memory for the per-node pools — the standard space-for-speed trade.

5. Distributed cache + consistent hashing (`app/cache_cluster.py` , `app/consistent_hash.py`)

The brief requires the **cache** to be split across multiple logical nodes, with **consistent hashing choosing which node owns a prefix key**. We run **3 Redis cache nodes**; a hash ring maps each prefix to exactly one node.

Read flow (`GET /suggest`): route the prefix to its owning node → ask only that node → **hit** returns instantly; **miss** computes from the Trie/Suggester and stores the result back on that node.

Why consistent hashing, not `hash(prefix) % N` ? Modulo remaps almost every key when N changes (3→4 nodes), cold-starting the whole cache. Consistent hashing places nodes and prefixes on a ring; a prefix is owned by the next node clockwise, so adding or removing one node moves only the keys in that node's arc — $\sim 1/N$ of them. Our `ch_demo` measures 22.9% remap when going 3→4 nodes (vs ~100% for modulo).

Virtual nodes (150/node): without them, single points on the ring give uneven arcs and one node gets overloaded; many virtual points smooth the distribution (measured $\approx 34/35/31\%$ across 3 nodes).

Routing key vs storage key: we *route* by prefix alone (both ranking modes for a prefix share a node), but *store* under `suggest:<mode>:<prefix>` so basic and recency results don't overwrite each other.

Expiry / invalidation: every entry has a TTL (basic 60s, recency 5s since it changes fast). On a batch flush we also actively delete the cached entries for every prefix of each changed query, so rankings refresh promptly; TTL is the backstop.

Honest trade-off: prefixes — not whole queries — are the cache key, so a prefix's completions are computed once and reused. The Trie itself is not sharded (it fits in memory for this scale); if it had to scale past one machine we'd shard it too and the `/suggest` miss path would scatter-gather across Trie shards.

6. Data storage & caching expectations

All satisfied: query-count data is held reliably in Postgres; suggestions go **cache** → **Trie** (cache first, primary index as fallback); the cache stores **per-prefix** suggestion results; entries **expire** (TTL) and are **invalidated** on writes; the cache is **distributed across 3 nodes** with **consistent hashing** owning each prefix key. `GET /cache/debug?prefix=` exposes the owner node and hit/miss for inspection.

7. Trending & recency-aware ranking (`app/activity.py` , `app/suggester.py`)

Basic ranking (60%): sort matching queries by **overall count**. The Trie pool is already sorted this way, so it's "take the top K".

Recency-aware ranking (enhanced +20%): combine popularity with recent activity:

```
combined_score = all_time_count + ALPHA * recent_count_in_window    (ALPHA = 50)
```

The four points the brief asks us to explain:

1. **How recent searches are tracked.** Time is bucketed into one-minute Redis sorted sets (`recent:<minute>`); each search does `ZINCRBY` on the current minute. "Last W minutes" = `ZUNIONSTORE` of the last W buckets.
2. **How recent activity affects ranking.** The `ALPHA * recent_count` term lifts queries being searched right now. We *display* the all-time count but *order* by the combined score, so a lower-count-but-surging query can sit above a quieter, higher count one. (Demonstrated in the logs: "iphone premium for beginners", count 936, jumps from rank 8 to rank 5 after 50 recent searches.)
3. **Avoiding permanent over-ranking of brief spikes.** `recent_count` comes from a *sliding window*; old buckets expire (TTL), so a spike's boost disappears after the window passes. A single running counter could never decay — this is why we window.
4. **How the cache is updated/invalidated when rankings change.** Recency entries use a short TTL (5s); plus, on each batch flush we actively invalidate the affected queries' prefixes. So changed rankings show up promptly.

Surfacing brand-new queries: the recency candidate set is the Trie pool **plus** recently-hot queries matching the prefix, so a surging query can appear even before its all-time count climbs into the pool.

Trade-offs (freshness vs latency vs complexity): recency mode does a couple of extra Redis reads and uses a short TTL, trading a little latency/freshness churn for much fresher results. Basic mode is cheapest and most cacheable. The same `/suggest` API serves both via `?ranking=`.

The **Trending panel** (`GET /trending`) uses the same window: the top-k of the recent union.

8. Batch writes (`app/batch_writer.py`)

Writing to Postgres on every `/search` means one tiny UPDATE per event — many round trips and lock contention.

Mechanism: `/search` increments an in-memory `Counter` (query → pending count). The buffer flushes when it holds `BATCH_MAX_SIZE` (100) distinct queries **or** every `BATCH_FLUSH_INTERVAL_SECONDS` (5s) — whichever first. A flush is **one bulk UPSERT**; repeated queries are aggregated (20 searches → `+20` in one statement). The DB returns new totals (`RETURNING`), which we use to update the Trie and invalidate caches.

Write reduction (measured): 240 events → 2 DB write statements = **120×** fewer writes (see `PERFORMANCE.md`).

Concurrency: an `asyncio.Lock` guards the buffer; we snapshot-and-swap under the lock, then do the DB write *outside* it so requests aren't blocked during I/O. The blocking `psycopg2` call runs in `asyncio.to_thread` so the event loop never freezes.

Failure trade-off (asked for explicitly): the buffer is in memory, so a crash before a flush loses at most one interval (~5s) of increments. For approximate, eventually-consistent popularity counts that's acceptable. For durability we'd persist the buffer to an append-only log or a queue (e.g. Kafka) and flush from there — the standard next step.

9. UI (frontend/)

Search box; suggestion dropdown that updates as you type (**debounced 120ms** to avoid unnecessary backend calls); submit on **Enter / Search button / clicking a suggestion**; the dummy `{"message": "Searched"}` response is displayed; a **trending** panel; **loading & error** states in the status tag; **keyboard navigation** (Arrow keys + Enter); and a basic/recency toggle so the ranking difference is visible live.

10. Non-functional

Runs locally with one `docker compose up . /suggest` is cache-fronted and in-memory (p95 $\approx$ 1.85ms). `/metrics` reports cache hit rate and DB read/write counts; `scripts/benchmark.py` reports p50/p95/p99; `scripts/ch_demo.py` logs consistent hashing behaviour. Code is split into one-job modules and documented.

11. What I'd do next (limitations, honestly stated)

- **Fault tolerance:** the Trie and cache are recoverable from Postgres on restart, but a production system would replicate Postgres and the cache nodes and persist the ring.
- **Durable writes:** append-only log in front of the batch buffer (see §8).
- **Fuzzy/typo tolerance:** currently strict prefix match; could add edit-distance.
- **Personalisation:** ranking is global; real systems blend personal history.

These are deliberate scope choices for a university-level build — all five graded concepts (data modeling, distributed cache + consistent hashing, trending/recency, batch writes, plus the working UI/APIs) are implemented, measured, and demonstrated.

5. Performance report

Performance Report

All numbers below were measured on the running stack (FastAPI + 3 Redis cache nodes + Redis-meta + Postgres) with a **100,000-query** dataset. Reproduce them yourself with the scripts in

backend/scripts/.

1. Suggestion latency (GET /suggest)

Measured with `python -m scripts.benchmark 3000`, which fires 3,000 requests across a realistic spread of prefixes (repeats included so the cache warms).

Metric	Value
Requests	3,000
Average	1.38 ms
p50	1.25 ms
p95	1.85 ms
p99	3.50 ms
Max	19.0 ms

Sub-2ms p95 — the in-memory Trie plus the distributed cache keep reads fast.

2. Cache hit rate

From the same run (GET /metrics):

Metric	Value
Cache hits	2,965
Cache misses	35
Hit rate	98.8%

The 35 misses are the first time each distinct prefix is seen (cold cache); every repeat is served from the owning cache node. This is the cache doing its job: popular prefixes almost never hit the Trie a second time within the TTL.

3. Database read/write counts

- **Reads:** 2 total — both at startup (count check + full load to build the Trie). Serving suggestions never reads the database; it reads the cache or the in-memory Trie. This is the whole point of the caching design.
- **Writes:** see below — writes only happen on batch flushes, never per request.

4. Batch-write reduction

Measured by firing **240 search submissions** across 5 distinct queries (POST /search) and reading GET /metrics:

Metric	Value
Search events received	240
Batch flushes (DB write statements)	2
Rows written	8
Write-reduction factor (events ÷ write calls)	120×

Without batching we would have issued **240** individual UPDATE statements. With batching we issued **2** bulk UPSERTs — a 120× reduction in database write operations. The reduction grows with traffic and with how often the same query repeats (repeats are aggregated in the buffer before the flush).

Failure trade-off: the buffer is in memory, so a crash before a flush loses at most one flush interval (default 5s) of increments. For approximate popularity counts that is acceptable; durability would come from writing the buffer to an append-only log or queue first (see `DESIGN.md §8`).

5. Consistent-hashing behaviour

From `python -m scripts.ch_demo` (10,000 prefixes, 150 virtual nodes/node):

Distribution across 3 nodes:

cache-0: 34.0% cache-1: 35.5% cache-2: 30.5%

After adding cache-3:

cache-0: 26.4% cache-1: 27.6% cache-2: 23.1% cache-3: 22.9%

Prefixes remapped: 22.9%

Adding a 4th node moved only **22.9%** of prefixes ($\approx 1/4$). Plain `hash % N` would have remapped close to 100%, cold-starting the entire cache. This is the property that makes the cache layer safely scalable.